

CHOFYAI STUDIO

08 · Flujo de datos

Documento	08-data-flow.md
Versión analizada	0.5.1 (commit f840055)
Fecha del análisis	2026-08-28
Fuente Markdown	docs/system-documentation/08-data-flow.md
Generado por	scripts/docs/build-pdf.mjs

Contenido

ChofyAI Studio · 08 · Flujo de datos

v0.5.1 (commit f840055) · 2026-08-28

1. Origen de los datos
2. Validación: quién comprueba qué
3. Transformaciones
4. Destino y consumo
5. Datos que salen del equipo
6. Qué se muestra al usuario y por qué canal
7. Flujo A · Arranque de la aplicación
8. Flujo B · Cambiar el Studio Home
9. Flujo C · Instalación con progreso
10. Flujo D · Arranque, salud y vista embebida
11. Flujo E · Descarga de un modelo declarado
12. Flujo F · Ejecución de un workflow HTTP
13. Flujo G · Detección y adopción de un huérfano
14. Dónde puede perderse o corromperse el dato
15. Datos personales y sensibles en tránsito

Estado: completo · Última revisión: 2026-08-27 · Versión analizada: 0.5.1 (commit f840055)

Este documento sigue el dato de punta a punta: de dónde entra, quién lo valida, en qué se convierte, dónde se guarda, quién lo consume y dónde puede perderse. Complementa [07 · Base de datos y persistencia](#), que describe los almacenes, y [06 · Explicación profunda del código](#), que describe la mecánica interna.

1. Origen de los datos

Origen	Qué aporta	Puerta de entrada
Usuario	Ruta del Studio Home, rutas de modelos/salidas/caché, destino de reubicación, id y YAML de workflows, entradas de workflow, archivos adjuntos	Formularios de <code>src/App.tsx</code>
Manifiestos del repositorio o del bundle	Catálogo de herramientas, puertos, scripts, criterios de instalación, modelos declarados	<code>collect_manifests</code>
<code>settings.json</code>	Configuración persistida	<code>load_settings</code>
<code>processes.json</code>	PIDs de la sesión anterior	<code>restore_registry</code>
Sistema operativo	CPU, memoria, disco, volúmenes, puertos ocupados, vida de un PID	<code>sysctl</code> , <code>vm_stat</code> , <code>top</code> , <code>df</code> , <code>lsof</code> , <code>kill -0</code>
Salida de los scripts	Progreso de instalación línea a línea	<code>stdout/stderr</code> capturados en <code>run_install_script</code>
Internet	Código de las herramientas, modelos, paquetes Python, último release publicado	<code>git</code> , <code>huggingface-cli</code> , <code>pip/uv</code> , <code>fetch</code> en <code>UpdateChecker</code>
Herramientas locales	Respuestas HTTP de inferencia	<code>fetch</code> en <code>runWorkflowStep</code> y el <code>iframe</code>

2. Validación: quién comprueba qué

Punto de control	Dónde	Qué valida
Formulario de workflow	<code>canRun</code> en <code>WorkflowRunner</code>	Que todas las entradas requeridas estén completas
Comprobación previa a instalar	<code>PreInstallCheck</code>	Espacio libre frente al tamaño estimado
Identificador de workflow	<code>validate_workflow_id</code>	No vacío, sin <code>/</code> , <code>\</code> ni <code>..</code> , sólo <code>[a-zA-Z0-9_-]</code>
Contenido de workflow	<code>save_workflow</code>	Que el YAML parsee, que la raíz sea un mapa y que existan <code>id</code> , <code>name</code> , <code>description</code> , <code>steps</code>
Ruta de modelo a borrar	<code>delete_tool_model</code>	Sin <code>..</code> , y tras <code>canonicalize</code> debe seguir dentro del directorio de modelos
Modelo a descargar	<code>download_tool_model</code>	Que el <code>repo_id</code> esté declarado en el manifiesto
Plataforma	<code>platform_supported</code>	Que la plataforma actual esté en <code>platforms:</code>
Integridad de la instalación	<code>installed_if</code> en <code>run_install_script</code> y en <code>start_tool</code>	Que existan los artefactos declarados
Destino de reubicación	<code>relocate_module</code>	Ruta absoluta, distinta del origen, destino vacío, padre escribible
Manifiestos	Job <code>validate-manifests</code> de <code>.github/workflows/ci.yml</code>	Campos obligatorios, categorías y runtimes válidos, <code>run</code> si hay <code>install_script</code>

Puntos **sin** validación, verificados en el código:

- `save_studio_home` acepta cualquier cadena: no comprueba que exista, que sea absoluta ni que sea escribible. El error aparece más tarde, al instalar.
- `save_path_settings` tampoco valida las rutas.
- El `run.command` del manifiesto se interpola tal cual en una shell.
- `list_workflows` devuelve el YAML convertido a JSON sin validarlo contra `WorkflowDef`: un workflow malformado llega al frontend y puede romper el render.
- El `id` de una entrada del marketplace se usa para construir `apps/<id>.yaml` sin sanear.

3. Transformaciones

Transformación	Función	De → A
YAML a estructura	<code>serde_yaml::from_str</code>	<code>apps/*.yaml</code> → <code>RawManifest</code>
Manifiesto a resumen	Cuerpo de <code>list_tools</code>	<code>RawManifest</code> + <code>settings</code> + estado del disco → <code>ToolSummary</code>
Resumen a modelo de interfaz	Serialización de Tauri	<code>ToolSummary</code> → <code>ToolManifest</code> (TypeScript)
Línea de log a estado de progreso	<code>parseInstallLine</code>	"Receiving objects: 47%..." → { phase, progressPct }
YAML a JSON	<code>list_workflows</code>	<code>serde_yaml::Value</code> → <code>serde_json::Value</code>
Bytes a texto legible	<code>fmtBytes</code> , <code>fmtElapsed</code>	7516192768 → 7.0 GB
Entrada de marketplace a manifiesto	<code>import_marketplace_tool</code>	<code>MarketplaceEntry</code> → <code>apps/<id>.yaml</code> mínimo
Formulario a YAML	<code>buildYaml</code>	Estado del constructor visual → texto YAML
Plantilla a valor	<code>substituteVars</code>	<code>{{ inputs.audio }}</code> → valor introducido
Páginas de memoria a bytes	<code>read_mem_used</code> + <code>parse_pages</code>	Salida de <code>vm_stat</code> → bytes usados

Detalle importante de `list_tools` : el campo `installed` **no** se guarda en ninguna parte; se calcula en cada llamada comprobando en disco cada entrada de `installed_if` . La verdad sobre si una herramienta está instalada es siempre el sistema de archivos, nunca un registro.

4. Destino y consumo

Dato	Se almacena en	Lo consume
Configuración	<code>settings.json</code>	Todo el backend, en cada comando
PIDs	Memoria + <code>processes.json</code>	<code>health_check_tool</code> , <code>list_orphan_ports</code> , <code>stop_tool</code>
Salida de instalación	<code><studio_home>/logs/<tool>-install.log</code>	<code>LogsViewer</code> , <code>open_tool_log</code>
Salida de ejecución	<code><studio_home>/logs/<tool>-run.log</code>	<code>LogsViewer</code>
Descarga de modelos	<code><studio_home>/logs/<tool>-model-download.log</code>	Lectura manual
Errores de interfaz	<code>storage/state/crash.log</code>	<code>read_crash_log</code>
Código y modelos	Árbol de <code>studio_home</code>	Las propias herramientas
Preferencias de interfaz	<code>localStorage</code>	<code>App</code> al montar

5. Datos que salen del equipo

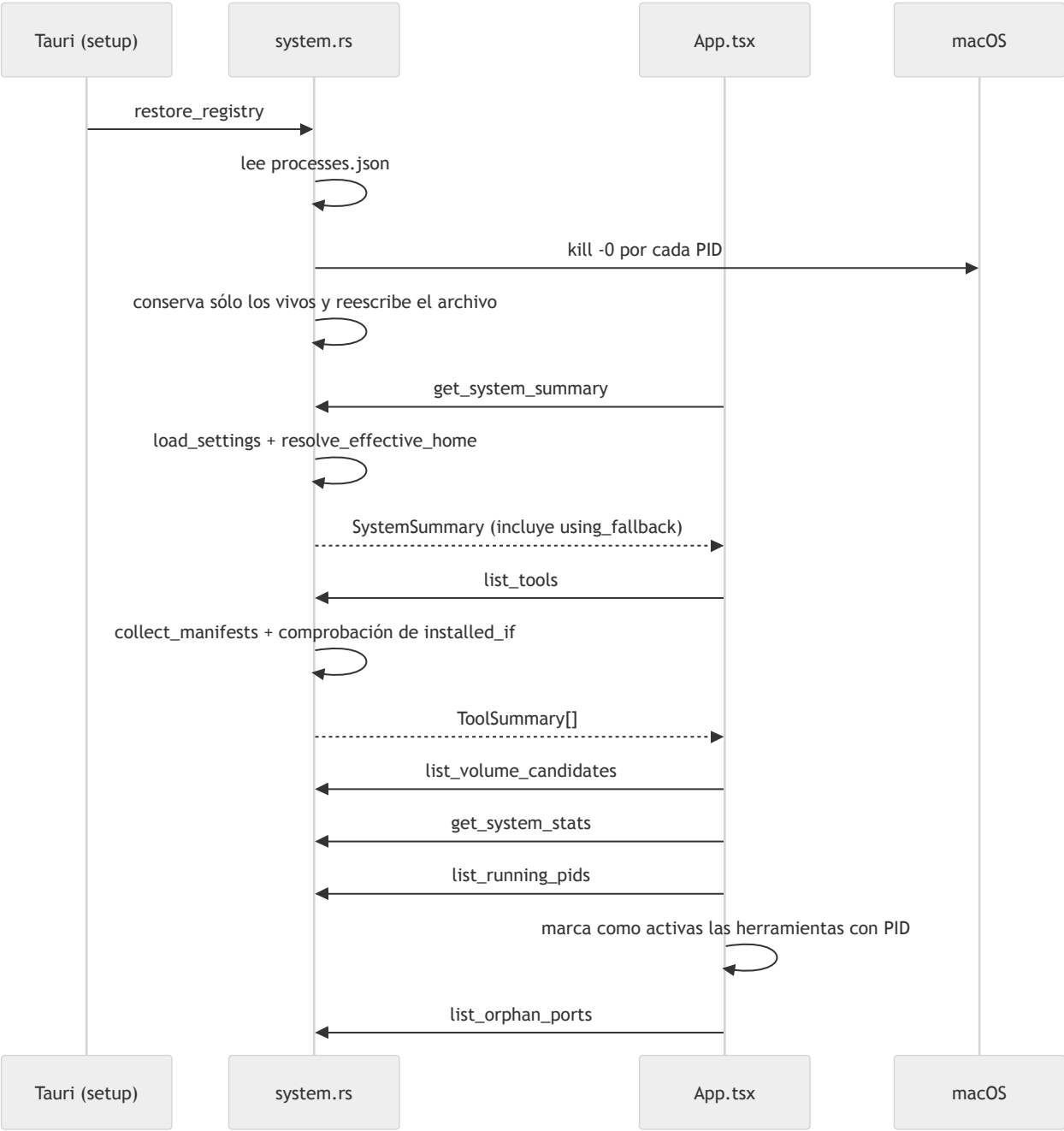
Destino	Cuándo	Qué se envía	Qué no se envía
GitHub (git)	Instalar o actualizar una herramienta	Petición de clonado o <code>pull</code> de un repositorio público	Nada del usuario
API de GitHub	Al abrir la aplicación (<code>UpdateChecker</code>)	Petición HTTPS al endpoint de releases	Ni versión local, ni identificadores
Hugging Face	Descargar un modelo	El <code>repo_id</code> solicitado	Contenido local
PyPI	Instalar dependencias Python	Nombres y versiones de paquetes	—
Homebrew	Instalar <code>ffmpeg</code> si falta	Petición de fórmula	—
jsDelivr	Sólo al generar los PDF de esta documentación	Petición del archivo de <code>mermaid</code>	—

Durante el **uso** de las herramientas no sale nada: la inferencia ocurre contra `127.0.0.1`. Es la propiedad central del producto y se sostiene en el código: los `run.command` de los manifiestos fijan `--host 127.0.0.1`, `--listen 127.0.0.1` o `GRADIO_SERVER_NAME=127.0.0.1`.

6. Qué se muestra al usuario y por qué canal

Canal	Contenido	Implementación
Tarjetas de herramienta	Estado, ruta, puerto, salud	<code>tools</code> + <code>health</code> en <code>App</code>
Barra de estado	CPU, memoria, disco, uptime, aviso de fallback	<code>StatusBar</code> con <code>SystemStats</code>
Cola de instalación	Fase, porcentaje, velocidad, tiempo, últimas líneas	<code>QueueItem</code> + <code>parseInstallLine</code>
Toasts	Éxitos y errores	<code>Toaster</code> y <code>notify</code>
Notificación nativa	Fin de instalación	<code>notify_macos</code> vía <code>osascript</code>
Visor de logs	Cola del archivo con filtro	<code>LogsViewer</code> + <code>read_tool_log</code>
Interfaz embebida	La UI de la propia herramienta	<code><iframe</code> <code>src="http://127.0.0.1:PORT/"></code>
Panel de modelos	Modelos en disco y declarados	<code>ModelsPanel</code>
Doctor	Salida cruda del script	<code>DoctorModal</code> + <code>run_doctor</code>

7. Flujo A · Arranque de la aplicación



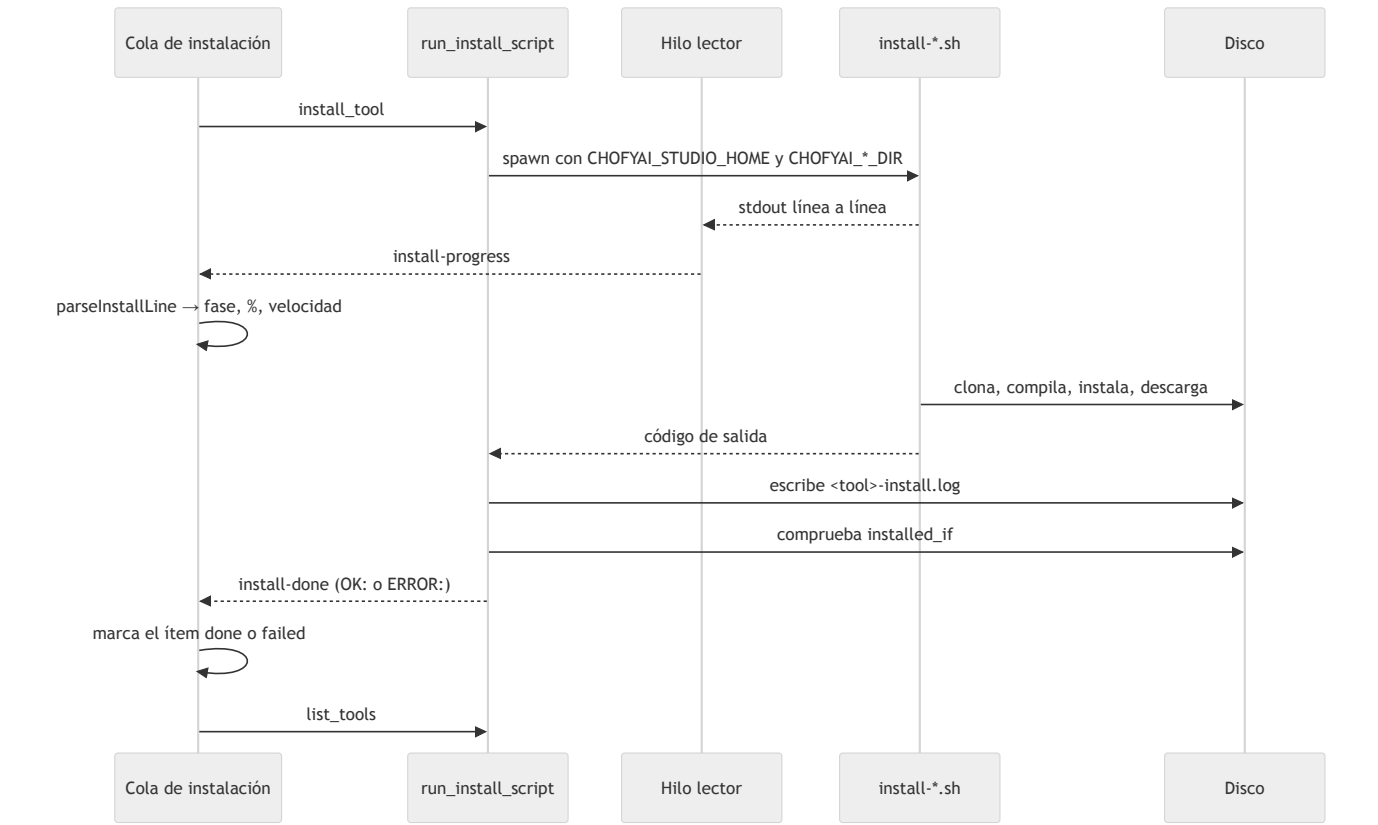
Si la aplicación se ejecuta sin backend (modo web), `tauriInvoke` devuelve `null` en todas esas llamadas y la interfaz se rellena con `fallbackTools` y un resumen deducido del `userAgent`. Es una simulación explícita, no un error.

8. Flujo B · Cambiar el Studio Home



Consecuencia práctica que conviene entender: cambiar el Studio Home **no mueve nada**. Las herramientas instaladas en la ruta anterior siguen allí y desaparecen de la lista porque `installed_if` ya no se cumple en la ruta nueva. Para mover una herramienta existe `relocate_module`.

9. Flujo C · Instalación con progreso

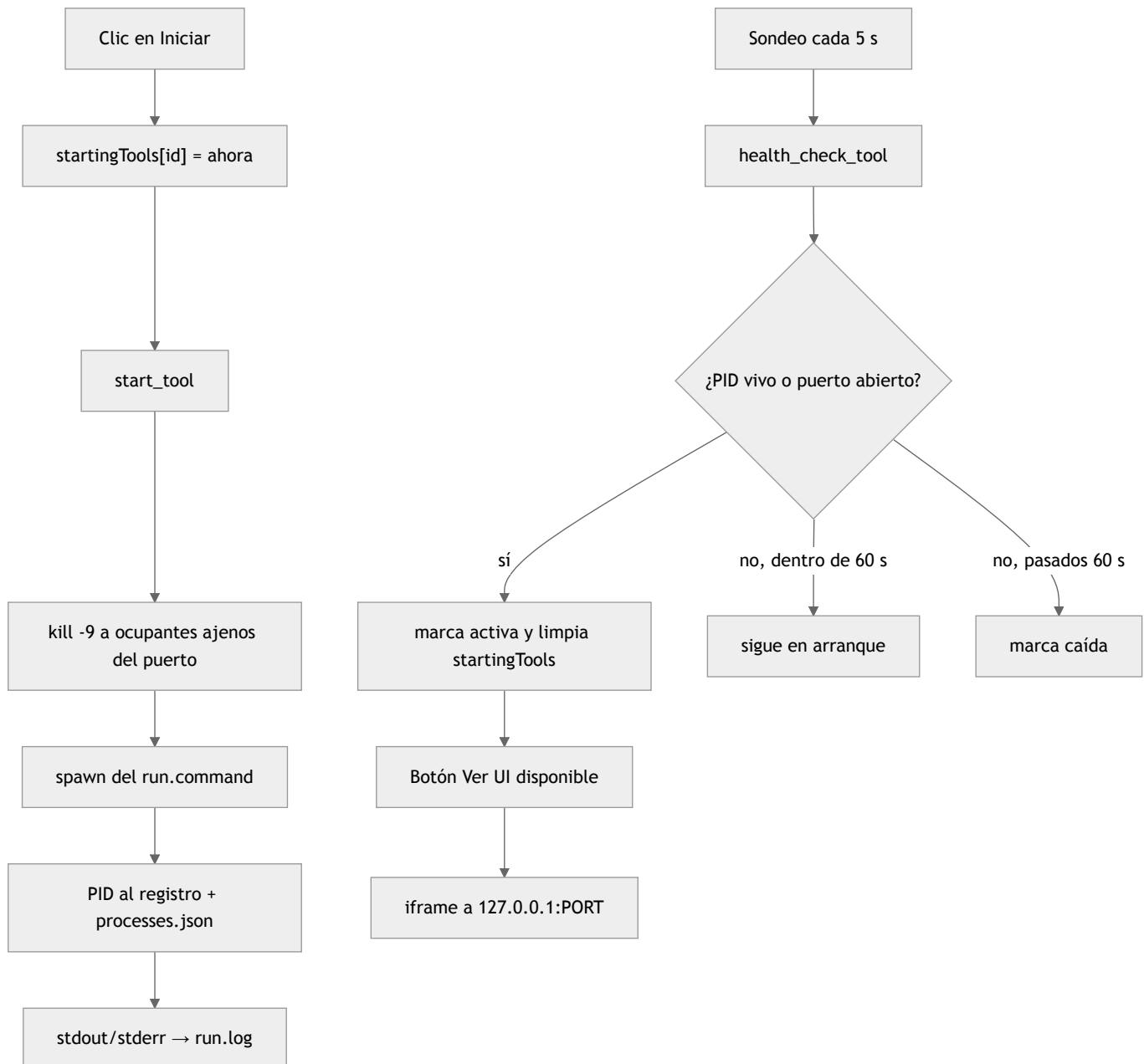


El dato viaja por dos caminos a la vez: como **evento** para el progreso y como **valor de retorno** al final. Si el frontend perdiera el evento `install-done`, el ítem quedaría en `installing` para siempre aunque el comando hubiese retornado; es un acoplamiento a tener presente.

10. Flujo D · Arranque, salud y vista embebida

ChofyAI Studio · 08 · Flujo de datos

v0.5.1 (commit f840055) · 2026-08-28



La ventana de 60 segundos existe porque herramientas como ComfyUI o FaceFusion tardan en abrir el puerto: sin ella, la interfaz declararía "caída" una herramienta que sólo estaba cargando modelos.

11. Flujo E · Descarga de un modelo declarado

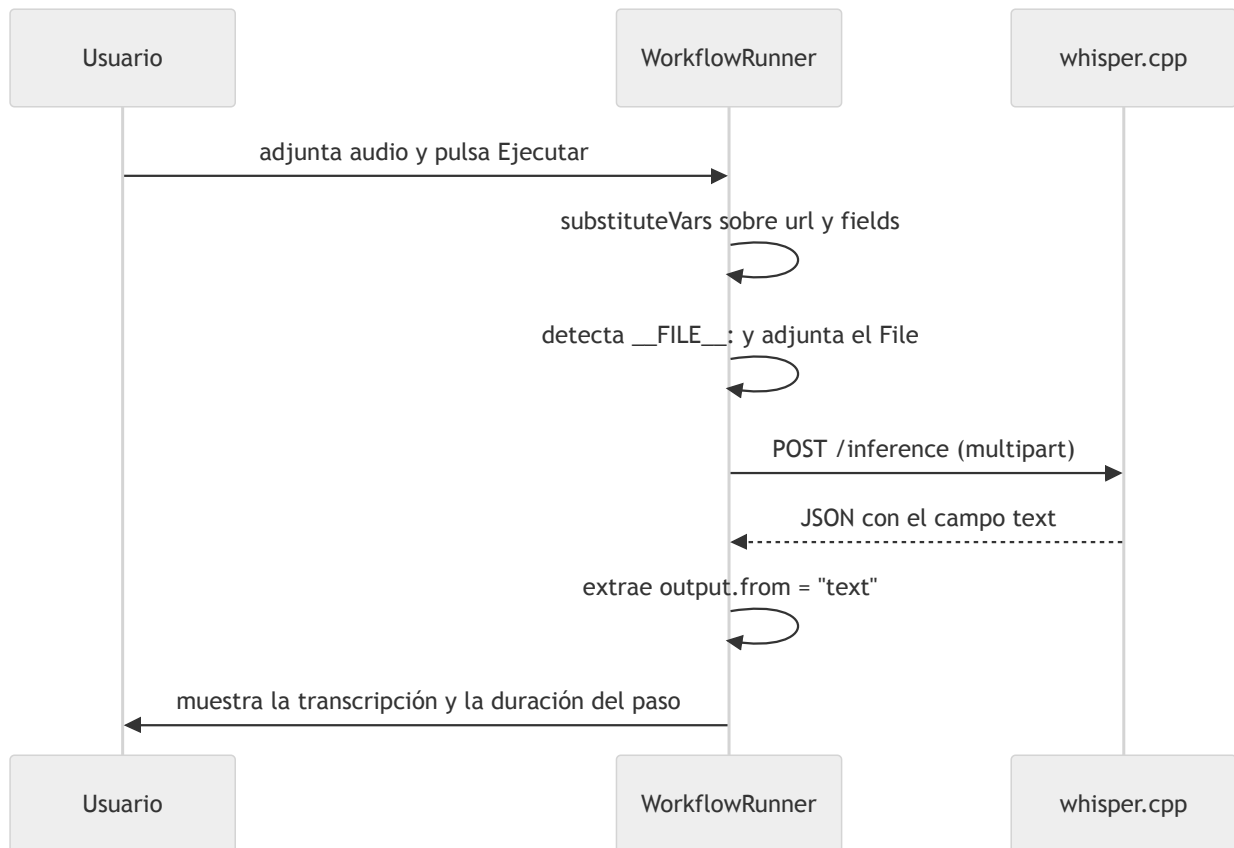
1. `ModelsPanel` llama a `list_declared_models`, que cruza `manifest.models` con el contenido de `<install_dir>/models`.
2. El usuario pulsa *Descargar* sobre uno ausente.
3. `download_tool_model` comprueba que ese `repo_id` esté declarado y ejecuta `scripts/mac/download-hf-model.sh <repo_id> <destino>`.
4. El script usa `huggingface-cli` si está disponible y, si no, `snapshot_download` de `huggingface_hub`, instalándolo en el Python detectado si hace falta.

5. Cada línea de salida se emite como `model-download-progress`.

6. Al terminar se escribe `<tool>-model-download.log` y se emite `model-download-done`. Copy to clipboard · [Sigue a @miguelsalazar](#) · [Ver commit f840055](#) · 2026-08-28

Punto de fragilidad: el script se instala `huggingface_hub` con `pip --user` sobre el Python detectado del sistema, no dentro del entorno virtual de la herramienta.

12. Flujo F · Ejecución de un workflow HTTP



Este flujo **no pasa por Rust**: el WebView habla directamente con el proceso local. Si la herramienta no está arrancada, `fetch` falla y el paso se marca con `Network: <mensaje>`. Los pasos con `type: stub` no ejecutan nada y devuelven su nota.

13. Flujo G · Detección y adopción de un huérfano

1. Cada 60 segundos, `list_orphan_ports` recorre los puertos declarados por los manifiestos.
2. Para cada uno ejecuta `lsof -nP -iTCP:<port> -sTCP:LISTEN -Fpc` y extrae PID y comando.
3. Si el PID no está en el registro, se reporta como huérfano.
4. La interfaz muestra un aviso; el usuario puede **adoptar** (el PID entra al registro y se persiste) o **matar** (`SIGTERM`).

Falso positivo posible y documentado: cualquier proceso ajeno que escuche en uno de esos puertos aparecerá como huérfano de la herramienta correspondiente, porque la identificación se hace **sólo por número de puerto**.

14. Dónde puede perderse o corromperse el dato

Situación	Consecuencia	Mitigación existente
El volumen se desmonta durante la instalación	Instalación a medias; el log puede no escribirse	Ninguna: se detecta después con <code>installed_if</code>
El proceso muere mientras se escribe <code>settings.json</code>	Archivo truncado	Ninguna; <code>load_settings</code> cae a los valores por defecto y se pierde la configuración
PID reciclado por el sistema	<code>health_check_tool</code> da por viva una herramienta que no lo está	Comprobación adicional del puerto
Otro proceso ocupa el puerto declarado	El pre-flight lo mata, o aparece como huérfano	Adoptar o matar desde la interfaz
<code>install-done</code> se pierde	El ítem de la cola queda en <code>installing</code>	Ninguna
Log muy grande	<code>read_tool_log</code> sólo devuelve la cola	Límite de 500 líneas por defecto
YAML de un manifiesto inválido	<code>collect_manifests</code> falla y ninguna herramienta aparece	Validación en CI, no en tiempo de ejecución
Descarga de modelo interrumpida	Directorio parcial que <code>list_declared_models</code> puede dar por presente	Ninguna: <code>present</code> sólo comprueba que el directorio no esté vacío
<code>iframe</code> apuntando a un puerto ocupado por otra aplicación	Se muestra contenido ajeno dentro de la ventana	Ninguna

15. Datos personales y sensibles en tránsito

Dato	Dónde aparece	Sale del equipo
Nombre de usuario y rutas del home	<code>settings.json</code> , logs, <code>crash.log</code> , mensajes de error	No
Nombres de volúmenes externos	<code>list_volume_candidates</code> , barra de estado	No
Archivos que el usuario adjunta a un workflow	Petición multipart a <code>127.0.0.1</code>	No
Prompts y textos introducidos	Cuerpo de la petición a la herramienta local	No
Contenido generado (audio, imagen, texto)	<code>outputs/</code> y respuestas HTTP locales	No
Trazas de error de la interfaz	<code>crash.log</code> , con hasta 4000 caracteres de stack	No

No se ha encontrado en el código analizado ningún envío de datos del usuario a un servicio remoto. El análisis de esta superficie está en [11 · Seguridad](#).

